

CMSC3621 Data Structures and Algorithms Lab

Lab Assignment 5

Overview:

This assignment is for the students to review and practice the basics of AVL Tree.

Details

In the attached .zip file, we provided a C++ project where an AVLTreeNode class has been implemented. But an AVLTree class is incomplete. An additional main.cpp is used to test the two classes.

You need to complete the AVLTree class in the avltree.h file. In particular, you only need to complete the insertion functionality of this class. The following member functions have been provided as samples / helpers :

- `void RotateLeft(AvlNode<T> * &node);`
- `void RotateRight(AvlNode<T> * &node);`
- `void CalculateTreeBalance(AvlNode<T>*& sub_root);`
- `int CalculateTreeHeight(AvlNode<T>*& sub_root);`

You need to complete other member functions, i.e. `insert()`, `RightBalanceAfterInsert()` and `LeftBalanceAfterInsert()`. You **cannot** modify any existing parts, you only can write your code under the promote “//Write your code here”.

We also provided a main.cpp file to test your AVLTree class. You can run your program to visualize whether you successfully create an AVL tree.

Submission

1) avltree.h file //Comments are needed to illustrate how do you implement the code
2) A report (in a PDF file), in the report you only need to provide two pieces of information:

- 2.1) screenshots and descriptions about how do you test your implementation and why your solution is correct according to the output
- 2.2) how do you use balance factor in your code to maintain your AVL tree

Important requirements:

- 1) Don't include any other files (that are not aforementioned).
- 2) Don't modify anything in the existing files / project, otherwise your code may not be able to run in the grader's environment.
- 3) You only can write your code at certain places which are indicated by the existing comments
- 4) Any violation against the rules above will lead to extra penalty. If you missed the report, your submission will not be evaluated.

Rubric

- 1) There are three incomplete functions in total. Partial credits may be given in terms of to what degree you can complete them.
 - 1.1) `insert()`: 40 pts
 - 1.2) `RightBalanceAfterInsert()`: 25 pts
 - 1.3) `LeftBalanceAfterInsert()`: 20 pts
- 2) Report: 15 pts